

Step 0: Environment Setup & Drive Mounting

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

Step 1: Dataset Overview & Patient ID Discovery

```
import os, glob

DATA_ROOT = "/content/drive/MyDrive/14 MS patients"

# list folders (patients)
patients = sorted([p for p in glob.glob(os.path.join(DATA_ROOT, "*")) if
print("Number of patient folders found:", len(patients))
print("\nFirst few patient folders:")
for p in patients[:10]:
    print(" -", os.path.basename(p))
```

Number of patient folders found: 17

First few patient folders:

- 00408990
- 00713519
- 00960975
- 00981157
- 01092669
- 01386182
- 01406050
- 01406050_1
- 01563167
- 01581148_2

Step 2: Train / Test Split Definition

```
# =====
# CHOOSE TRAIN / TEST PATIENTS
# =====

TRAIN_IDS = [
    "00713519",
    "01092669",
    "01386182",
    "01406050",
```

```

        "01722254",
        "01406050_1",
        "01563167",
        "01581148_2",
        "01581148_4",
        "01668999",
    ]

    # keep SAME test patients
    TEST_IDS = ["00960975", "00981157"]

```

Verifying the ids

```

print("Training IDs:", TRAIN_IDS)
print("Testing IDs:", TEST_IDS)

```

```

Training IDs: ['00713519', '01092669', '01386182', '01406050', '01722254', '01406050_1', '01563167', '01581148_2', '01581148_4', '01668999']
Testing IDs: ['00960975', '00981157']

```

Step 3: Inspect Selected Patient Folders

```

# =====
# STEP 3 – Inspect chosen patients
# Identify baseline / follow-up / label files
# =====

import os, re, glob
from pathlib import Path

#  EDIT THESE
#  EDIT THESE (use the same split as Step 2)
#  EDIT THESE (use the same split as Step 2)
TRAIN_IDS = [
    "00713519",
    "01092669",
    "01386182",
    "01406050",
    "01722254",
    "01406050_1",
    "01563167",
    "01581148_2",
    "01581148_4",
    "01668999",
]

TEST_IDS = ["00960975", "00981157"]

```

```
DATA_ROOT = "/content/drive/MyDrive/14 MS patients"
```

```
def parse_date_from_name(name: str):
```

```
    """
```

```
    Extract an 8-digit date (YYYYMMDD) from filename if present.
```

```
    Example: 00713519_20181002.nii.gz -> 20181002
```

```
    Returns int date or None.
```

```
    """
```

```
    m = re.search(r"(19|20)\d{6}", name)
```

```
    return int(m.group(0)) if m else None
```

```
def classify_files(patient_dir: str):
```

```
    """
```

```
    Returns:
```

```
    - baseline_path (earliest dated nii)
```

```
    - followup_path (latest dated nii)
```

```
    - label_path (best guess by name)
```

```
    - other_nii (everything else)
```

```
    - mats (mat files)
```

```
    """
```

```
nii = sorted(glob.glob(os.path.join(patient_dir, "*.nii.gz")))
```

```
mats = sorted(glob.glob(os.path.join(patient_dir, "*.mat")))
```

```
# candidate label by name
```

```
label_candidates = []
```

```
for f in nii:
```

```
    bn = os.path.basename(f).lower()
```

```
    if "label" in bn or "seg" in bn or "mask" in bn or "gt" in bn:
```

```
        label_candidates.append(f)
```

```
# exclude label candidates from flair candidates
```

```
flair_candidates = [f for f in nii if f not in label_candidates]
```

```
# try to pick baseline/follow-up based on date in filename
```

```
dated = []
```

```
for f in flair_candidates:
```

```
    d = parse_date_from_name(os.path.basename(f))
```

```
    if d is not None:
```

```
        dated.append((d, f))
```

```
baseline = followup = None
```

```
if len(dated) >= 2:
```

```
    dated.sort(key=lambda x: x[0])
```

```
    baseline = dated[0][1]
```

```
    followup = dated[-1][1]
```

```
else:
```

```
    # fallback: just take first two as baseline/followup (not ideal, but pr
```

```

        if len(flair_candidates) >= 2:
            baseline, followup = flair_candidates[0], flair_candidates[1]

# pick label: prefer exact "Segmentation-label"
label = None
if label_candidates:
    exact = [f for f in label_candidates if "segmentation-label" in os.path.basename(f)]
    label = exact[0] if exact else label_candidates[0]

other_nii = [f for f in nii if f not in [baseline, followup, label] and f != None]

return baseline, followup, label, other_nii, mats

def print_patient_summary(pid: str):
    pdir = os.path.join(DATA_ROOT, pid)
    print("\n" + "="*80)
    print("Patient:", pid)
    print("Folder:", pdir)

    if not os.path.isdir(pdir):
        print("❌ Folder not found.")
        return

    baseline, followup, label, other_nii, mats = classify_files(pdir)

    # List all files first
    all_files = sorted([p.name for p in Path(pdir).glob("*")])
    print("\nAll files in folder:")
    for f in all_files:
        print("  ", f)

    print("\n✅ Detected files:")
    if baseline:
        print("    Baseline   :", os.path.basename(baseline))
    else:
        print("    Baseline   : ⚠️ not detected")

    if followup:
        print("    Follow-up   :", os.path.basename(followup))
    else:
        print("    Follow-up   : ⚠️ not detected")

    if label:
        print("    Label/GT    :", os.path.basename(label))
    else:
        print("    Label/GT    : ⚠️ not detected (no file with label/seg/mask/gt i")

    if mats:
        print("\nMAT files (registration?):")
        for m in mats:

```

```

        print("  ", os.path.basename(m))
    else:
        print("\nMAT files: none")

    if other_nii:
        print("\nOther NIfTI files (maybe registered volumes, masks, etc.):")
        for f in other_nii:
            print("  ", os.path.basename(f))
    else:
        print("\nOther NIfTI files: none")

# Run summaries for your split
print("Training patients:", TRAIN_IDS)
print("Testing patients:", TEST_IDS)

for pid in TRAIN_IDS + TEST_IDS:
    print_patient_summary(pid)

```

```

Training patients: ['00713519', '01092669', '01386182', '01406050', '01722254']
Testing patients: ['00960975', '00981157']

```

```

=====

```

Patient: 00713519

Folder: /content/drive/MyDrive/14 MS patients/00713519

All files in folder:

- 00713519_20181002.nii.gz
- 00713519_20190830.nii.gz
- 00713519_20190830_to_FLAIR_1.nii.gz
- FLAIR_2_to_FLAIR_1.mat
- Segmentation-label.nii.gz

✓ Detected files:

Baseline : 00713519_20181002.nii.gz

Follow-up : 00713519_20190830_to_FLAIR_1.nii.gz

Label/GT : Segmentation-label.nii.gz

MAT files (registration?):

- FLAIR_2_to_FLAIR_1.mat

Other NIfTI files (maybe registered volumes, masks, etc.):

- 00713519_20190830.nii.gz

```

=====

```

Patient: 01092669

Folder: /content/drive/MyDrive/14 MS patients/01092669

All files in folder:

- 01092669_20171206.nii.gz
- 01092669_20190309.nii.gz
- 01092669_20190309_to_FLAIR_1.nii.gz

```

- FLAIR_2_to_FLAIR_1.mat
- Segmentation-label.nii.gz

✅ Detected files:
Baseline : 01092669_20171206.nii.gz
Follow-up : 01092669_20190309_to_FLAIR_1.nii.gz
Label/GT : Segmentation-label.nii.gz

MAT files (registration?):
- FLAIR_2_to_FLAIR_1.mat

Other NIFTI files (maybe registered volumes, masks, etc.):
- 01092669_20190309.nii.gz

=====
Patient: 01386182
Folder: /content/drive/MyDrive/14 MS patients/01386182

All files in folder:
- 01386182_20180213.nii.gz
- 01386182_20190115.nii.gz
- 01386182_20190115_to_FLAIR_1.nii.gz
- FLAIR_2_to_FLAIR_1.mat
- Segmentation-label.nii.gz

```

Sanity Check: Geometry & GT Alignment (Before Preprocessing)

```

import os
import nibabel as nib
import numpy as np

DATA_ROOT = "/content/drive/MyDrive/14 MS patients"
TRAIN_IDS = [
    "00713519",
    "01092669",
    "01386182",
    "01406050",
    "01722254",
    "01406050_1",
    "01563167",
    "01581148_2",
    "01581148_4",
    "01668999",
]
TEST_IDS = ["00960975", "00981157"]

def info(path):
    img = nib.load(path)
    shape = img.shape
    zooms = img.header.get_zooms()[ :3]

```

```

        affine = img.affine
        return shape, zooms, affine

def affine_close(a, b, tol=1e-3):
    return np.allclose(a, b, atol=tol)

def print_geom(pid):
    pdir = os.path.join(DATA_ROOT, pid)

    # expected filenames in your dataset
    files = sorted([f for f in os.listdir(pdir) if f.endswith(".nii.gz")])
    baseline = [f for f in files if "_to_FLAIR_1" not in f and "Segmentation-label"]
    # better: pick earliest date as baseline (works with your naming)
    dated = sorted([(int(f.split("_")[1].split(".")[0]), f) for f in files if f.rsplit(".", 1)[0].isdigit()])
    baseline = dated[0][1]
    followup_original = dated[-1][1]
    followup_to_f1 = [f for f in files if "_to_FLAIR_1" in f][0]
    label = [f for f in files if "Segmentation-label" in f][0]

    paths = {
        "baseline": os.path.join(pdir, baseline),
        "followup_original": os.path.join(pdir, followup_original),
        "followup_to_FLAIR_1": os.path.join(pdir, followup_to_f1),
        "label": os.path.join(pdir, label)
    }

    print("\n" + "="*90)
    print("Patient:", pid)
    for k, p in paths.items():
        sh, z, aff = info(p)
        print(f"{k:18s} | shape={sh} | zooms={z} | file={os.path.basename(p)}")

    # compare geometry
    b_aff = nib.load(paths["baseline"]).affine
    b_shape = nib.load(paths["baseline"]).shape

    f1_aff = nib.load(paths["followup_to_FLAIR_1"]).affine
    f1_shape = nib.load(paths["followup_to_FLAIR_1"]).shape

    l_aff = nib.load(paths["label"]).affine
    l_shape = nib.load(paths["label"]).shape

    print("\nGeometry checks (compared to BASELINE):")
    print("followup_to_FLAIR_1 same shape? ", f1_shape == b_shape)
    print("followup_to_FLAIR_1 same affine? ", affine_close(f1_aff, b_aff))
    print("label same shape? ", l_shape == b_shape)
    print("label same affine? ", affine_close(l_aff, b_aff))

import re
for pid in TRAIN_IDS + TEST_IDS:
    print_geom(pid)

```

```
=====
Patient: 00713519
baseline           | shape=(326, 256, 256) | zooms=(np.float32(0.56000054), np
followup_original  | shape=(295, 256, 256) | zooms=(np.float32(0.56), np.float
followup_to_FLAIR_1 | shape=(326, 256, 256) | zooms=(np.float32(0.56000054), n
label              | shape=(326, 256, 256) | zooms=(np.float32(0.56000054), np
```

Geometry checks (compared to BASELINE):

followup_to_FLAIR_1 same shape? True

followup_to_FLAIR_1 same affine? True

label same shape? True

label same affine? True

```
=====
Patient: 01092669
baseline           | shape=(290, 256, 256) | zooms=(np.float32(0.55999786), np
followup_original  | shape=(256, 256, 252) | zooms=(np.float32(0.978113), np.f
followup_to_FLAIR_1 | shape=(290, 256, 256) | zooms=(np.float32(0.55999786), n
label              | shape=(290, 256, 256) | zooms=(np.float32(0.55999786), np
```

Geometry checks (compared to BASELINE):

followup_to_FLAIR_1 same shape? True

followup_to_FLAIR_1 same affine? True

label same shape? True

label same affine? True

```
=====
Patient: 01386182
baseline           | shape=(326, 256, 256) | zooms=(np.float32(0.56001043), np
followup_original  | shape=(295, 256, 256) | zooms=(np.float32(0.5600016), np.
followup_to_FLAIR_1 | shape=(326, 256, 256) | zooms=(np.float32(0.56001043), n
label              | shape=(326, 256, 256) | zooms=(np.float32(0.56001043), np
```

Geometry checks (compared to BASELINE):

followup_to_FLAIR_1 same shape? True

followup_to_FLAIR_1 same affine? True

label same shape? True

label same affine? True

```
=====
Patient: 01406050
baseline           | shape=(144, 512, 512) | zooms=(np.float32(1.0), np.float3
followup_original  | shape=(295, 256, 256) | zooms=(np.float32(0.5600014), np.
followup_to_FLAIR_1 | shape=(144, 512, 512) | zooms=(np.float32(1.0), np.float
label              | shape=(144, 512, 512) | zooms=(np.float32(1.0), np.float3
```

Geometry checks (compared to BASELINE):

followup_to_FLAIR_1 same shape? True

followup_to_FLAIR_1 same affine? True

label same shape? True

label same affine? True

```
=====
Patient: 01722254
baseline      | shape=(326, 256, 256) | zooms=(np.float32(0.5599937), np.f
followup_original | shape=(326, 256, 256) | zooms=(np.float32(0.559993), np.f
```

Preprocessing

Step 4: Preprocessing – Global Target Shape Definition

```
import os, re, glob
import nibabel as nib
import numpy as np

DATA_ROOT = "/content/drive/MyDrive/14 MS patients"
TARGET_SPACING = (0.5, 0.5, 0.5)

# Explicitly ignore known bad patients
IGNORE_IDS = {"00408990"}

def parse_date_from_name(name: str):
    m = re.search(r"(19|20)\d{6}", name)
    return int(m.group(0)) if m else None

def pick_followup_original(patient_dir: str):
    nii = [f for f in glob.glob(os.path.join(patient_dir, "*.nii.gz"))]
    candidates = []
    for f in nii:
        bn = os.path.basename(f).lower()
        if "_to_flair_1" in bn:
            continue
        if "segmentation-label" in bn or "label" in bn:
            continue
        d = parse_date_from_name(os.path.basename(f))
        if d is not None:
            candidates.append((d, f))
    if not candidates:
        return None
    candidates.sort(key=lambda x: x[0])
    return candidates[-1][1]

def estimate_resampled_shape(shape, zooms, target_spacing):
    return tuple(
        int(np.round(shape[i] * (zooms[i] / target_spacing[i])))
        for i in range(3)
    )

patient_dirs = sorted([
    os.path.join(DATA_ROOT, d)
    for d in os.listdir(DATA_ROOT)
```

```

    if os.path.isdir(os.path.join(DATA_ROOT, d)) and d not in IGNORE_IDS
])

max_shape = (0, 0, 0)
valid_patients = []

for pdir in patient_dirs:
    pid = os.path.basename(pdir)
    fu = pick_followup_original(pdir)

    if fu is None:
        print(f"⚠ Skipping {pid}: no valid follow-up found")
        continue

    try:
        img = nib.load(fu)
        shape = img.shape[:3]
        zooms = img.header.get_zooms()[:3]
    except Exception as e:
        print(f"⚠ Skipping {pid}: could not load image ({e})")
        continue

    est = estimate_resampled_shape(shape, zooms, TARGET_SPACING)
    max_shape = tuple(max(max_shape[i], est[i]) for i in range(3))
    valid_patients.append((pid, shape, zooms, est))

print("\n=====")
print("Valid patients used:", len(valid_patients))
print("TARGET_SPACING:", TARGET_SPACING)
print("TARGET_SHAPE (for padding):", max_shape)

print("\nLargest resampled volumes:")
for pid, shape, zooms, est in sorted(
    valid_patients,
    key=lambda x: x[3][0]*x[3][1]*x[3][2],
    reverse=True
)[:5]:
    print(f"{pid}: original={shape}, zooms={tuple(float(z) for z in zooms)} → €
```

```

⚠ Skipping design1_models: no valid follow-up found
⚠ Skipping model_coat_seg: no valid follow-up found
⚠ Skipping processed: no valid follow-up found
```

```
=====
```

```

Valid patients used: 13
TARGET_SPACING: (0.5, 0.5, 0.5)
TARGET_SHAPE (for padding): (501, 501, 500)
```

```
Largest resampled volumes:
```

```

01888003: original=(326, 256, 256), zooms=(0.6100000143051147, 0.9765625, 0.9765
00960975: original=(310, 256, 256), zooms=(0.6000075340270996, 0.9765625, 0.9765
```

```
00981157: original=(326, 256, 256), zooms=(0.5600026249885559, 0.9765625, 0.9765
01406050_1: original=(256, 256, 326), zooms=(0.9765625, 0.9765625, 0.56000125408
01563167: original=(326, 256, 256), zooms=(0.5600038170814514, 0.9765625, 0.9765
```

Step 5.1(Training): Preprocessing Execution (NeuroPoly-style, Per Patient)

- Preprocess ONE training patient
- This will do, for TRAIN_ID=00713519:
- Load baseline + follow-up (original) + GT
- Register baseline → follow-up (follow-up reference, like NeuroPoly)
- Resample to 0.5mm (both images, plus label with nearest-neighbor)
- Normalize intensities
- (Optional) pad (we'll implement padding in a way that won't blow memory)

```
!pip -q install SimpleITK
```

52.6/52.6 MB 47.3 MB/s eta 0:00:00

```
# =====
# Preprocess TRAIN patients (5 IDs)
# NeuroPoly-style: baseline registered to followup, resample to 0.5mm, z-score
# =====

import os, re, glob
import numpy as np
import SimpleITK as sitk

# ----- PATHS -----
DATA_ROOT = "/content/drive/MyDrive/14 MS patients"
OUT_ROOT = "/content/drive/MyDrive/ms_preprocessed"
os.makedirs(OUT_ROOT, exist_ok=True)

# ----- SPLIT (TRAIN ONLY HERE) -----
# ----- SPLIT (TRAIN ONLY HERE) -----
TRAIN_IDS = [
    "00713519",
    "01092669",
    "01386182",
    "01406050",
    "01722254",
    "01406050_1",
    "01563167",
    "01581148_2",
    "01581148_4",
```

```

        "01668999",
    ]

TARGET_SPACING = (0.5, 0.5, 0.5) # (x,y,z) in SimpleITK

# ----- HELPERS -----
def parse_date(name: str):
    m = re.search(r"(19|20)\d{6}", name)
    return int(m.group(0)) if m else None

def get_patient_paths(pid: str):
    pdir = os.path.join(DATA_ROOT, pid)
    if not os.path.isdir(pdir):
        raise FileNotFoundError(f"Patient folder not found: {pdir}")

    nii = sorted(glob.glob(os.path.join(pdir, "*.nii.gz")))
    if len(nii) == 0:
        raise FileNotFoundError(f"No .nii.gz files found in {pdir}")

    # label
    label_list = [f for f in nii if "segmentation-label" in os.path.basename(f)]
    if not label_list:
        # fallback candidates
        label_list = [f for f in nii if any(k in os.path.basename(f).lower() for k in ["segmentation-label", "label", "seg", "mask"])]
    if not label_list:
        raise FileNotFoundError(f"No label/GT file found for patient {pid}")
    label = label_list[0]

    # originals (exclude *_to_FLAIR_1 and label)
    orig = []
    for f in nii:
        bn = os.path.basename(f).lower()
        if "segmentation-label" in bn or "label" in bn or "seg" in bn or "mask" in bn:
            continue
        if "_to_flair_1" in bn:
            continue
        d = parse_date(os.path.basename(f))
        if d is not None:
            orig.append((d, f))

    if len(orig) < 2:
        raise FileNotFoundError(f"Could not find 2 dated baseline/followup volumes")

    orig.sort(key=lambda x: x[0])
    baseline = orig[0][1]
    followup = orig[-1][1]
    return baseline, followup, label

def sitk_read(path: str):
    return sitk.ReadImage(path)

```

```

def make_reference_with_spacing(ref_img, out_spacing):
    # Create a new reference grid with desired spacing, preserving physical size
    in_spacing = ref_img.GetSpacing()
    in_size = ref_img.GetSize()
    out_size = [
        int(np.round(in_size[i] * (in_spacing[i] / out_spacing[i])))
        for i in range(3)
    ]
    out_ref = sitk.Image(out_size, ref_img.GetPixelID())
    out_ref.SetSpacing(out_spacing)
    out_ref.SetDirection(ref_img.GetDirection())
    out_ref.SetOrigin(ref_img.GetOrigin())
    return out_ref

def register_baseline_to_followup(baseline_img, followup_img):
    # Affine registration (Mattes MI + multi-resolution)
    initial = sitk.CenteredTransformInitializer(
        followup_img, baseline_img, sitk.AffineTransform(3),
        sitk.CenteredTransformInitializerFilter.GEOMETRY
    )

    reg = sitk.ImageRegistrationMethod()
    reg.SetMetricAsMattesMutualInformation(numberOfHistogramBins=32)
    reg.SetMetricSamplingStrategy(reg.RANDOM)
    reg.SetMetricSamplingPercentage(0.01)
    reg.SetInterpolator(sitk.sitkLinear)

    reg.SetOptimizerAsGradientDescent(
        learningRate=1.0,
        numberOfIterations=200,
        convergenceMinimumValue=1e-6,
        convergenceWindowSize=10
    )
    reg.SetOptimizerScalesFromPhysicalShift()

    reg.SetShrinkFactorsPerLevel([4, 2, 1])
    reg.SetSmoothingSigmasPerLevel([2, 1, 0])
    reg.SmoothingSigmasAreSpecifiedInPhysicalUnitsOn()

    reg.SetInitialTransform(initial, inplace=False)
    final_tfm = reg.Execute(followup_img, baseline_img)
    return final_tfm

def simple_foreground_mask(img):
    # quick mask: non-zero / low-percentile threshold
    arr = sitk.GetArrayFromImage(img).astype(np.float32)
    nz = arr[arr > 0]
    if nz.size == 0:
        m = arr != 0
    else:

```

```

        thr = np.percentile(nz, 5)
        m = arr > thr
    out = sitk.GetImageFromArray(m.astype(np.uint8))
    out.CopyInformation(img)
    return out

def zscore_in_mask(vol, mask):
    v = sitk.GetArrayFromImage(vol).astype(np.float32)
    m = sitk.GetArrayFromImage(mask) > 0
    if m.sum() == 0:
        m = v != 0
    vals = v[m]
    mu, sd = float(vals.mean()), float(vals.std() + 1e-6)
    v = (v - mu) / sd
    out = sitk.GetImageFromArray(v)
    out.CopyInformation(vol)
    return out

# =====
# RUN PREPROCESSING (TRAIN IDS)
# =====
for pid in TRAIN_IDS:
    print("\n" + "="*80)
    print("Preprocessing TRAIN patient:", pid)

    try:
        baseline_p, followup_p, label_p = get_patient_paths(pid)
        print("Baseline:", os.path.basename(baseline_p))
        print("Followup:", os.path.basename(followup_p))
        print("Label  :", os.path.basename(label_p))

        b = sitk_read(baseline_p)
        f = sitk_read(followup_p)
        gt = sitk_read(label_p)

        # 1) Register baseline -> followup space
        tfm = register_baseline_to_followup(b, f)
        b_reg = sitk.Resample(b, f, tfm, sitk.sitkLinear, 0.0, b.GetPixelID())

        # 2) Build reference grid with desired spacing in FOLLOWUP space
        ref05 = make_reference_with_spacing(f, TARGET_SPACING)

        # 3) Resample followup and registered baseline to reference grid
        f_05 = sitk.Resample(f, ref05, sitk.Transform(), sitk.sitkLinear, 0.0,
                             b_05 = sitk.Resample(b_reg, ref05, sitk.Transform(), sitk.sitkLinear, 0.0,

        # 4) Warp GT baseline->followup then resample to reference grid (nearest)
        gt_to_followup = sitk.Resample(gt, f, tfm, sitk.sitkNearestNeighbor, 0,
        gt_05 = sitk.Resample(gt_to_followup, ref05, sitk.Transform(), sitk.sitkNearestNeighbor, 0,

        # 5) Normalization (z-score within foreground of followup)

```

```

mask = simple_foreground_mask(f_05)
f_norm = zscore_in_mask(f_05, mask)
b_norm = zscore_in_mask(b_05, mask)

# 6) Save outputs
out_dir = os.path.join(OUT_ROOT, pid)
os.makedirs(out_dir, exist_ok=True)

sitk.WriteImage(b_norm, os.path.join(out_dir, "baseline_0p5_registered_
sitk.WriteImage(f_norm, os.path.join(out_dir, "followup_0p5_norm.nii.gz
sitk.WriteImage(gt_05, os.path.join(out_dir, "label_0p5_in_followup_s

print("Saved to:", out_dir)
print("Done )

except Exception as e:
    print(f" Failed for {pid}: {e}")

```

```

=====
Preprocessing TRAIN patient: 00713519
Baseline: 00713519_20181002.nii.gz
Followup: 00713519_20190830.nii.gz
Label   : Segmentation-label.nii.gz
Saved to: /content/drive/MyDrive/ms_preprocessed/00713519
Done 

=====
Preprocessing TRAIN patient: 01092669
Baseline: 01092669_20171206.nii.gz
Followup: 01092669_20190309.nii.gz
Label   : Segmentation-label.nii.gz
Saved to: /content/drive/MyDrive/ms_preprocessed/01092669
Done 

=====
Preprocessing TRAIN patient: 01386182
Baseline: 01386182_20180213.nii.gz
Followup: 01386182_20190115.nii.gz
Label   : Segmentation-label.nii.gz
Saved to: /content/drive/MyDrive/ms_preprocessed/01386182
Done 

=====
Preprocessing TRAIN patient: 01406050
Baseline: 01406050_20170912.nii.gz
Followup: 01406050_20180430.nii.gz
Label   : Segmentation-label.nii.gz
Saved to: /content/drive/MyDrive/ms_preprocessed/01406050
Done 

=====
Preprocessing TRAIN patient: 01722254

```

```
Baseline: 01722254_20190221.nii.gz
Followup: 01722254_20190801.nii.gz
Label : Segmentation-label.nii.gz
Saved to: /content/drive/MyDrive/ms_preprocessed/01722254
Done 
```

```
=====
Preprocessing TRAIN patient: 01406050_1
Baseline: 01406050_20180430.nii.gz
Followup: 01406050_20190325.nii.gz
Label : Segmentation-label.nii.gz
Saved to: /content/drive/MyDrive/ms_preprocessed/01406050_1
Done 
```

```
=====
Preprocessing TRAIN patient: 01563167
Baseline: 01563167_20171113.nii.gz
Followup: 01563167_20180625.nii.gz
Label : Segmentation-label_2.nii.gz
Saved to: /content/drive/MyDrive/ms_preprocessed/01563167
Done 
```

We must sanity-check that:

- baseline_registered_to_followup and followup are aligned
- label is in the same follow-up space (very important)
- The label overlaps plausible bright lesion regions (at least visually)
- So the next step is: visualize a few slices (like your instructor's GT vs prediction style, but now as an overlay on the follow-up).

```
import os
import numpy as np
import nibabel as nib
import matplotlib.pyplot as plt

#  Put your 5 training IDs here
TRAIN_IDS = [
    "00713519",
    "01092669",
    "01386182",
    "01406050",
    "01722254",
    "01406050_1",
    "01563167",
    "01581148_2",
    "01581148_4",
    "01668999",
]
```

```

BASE_PREP_ROOT = "/content/drive/MyDrive/ms_preprocessed"

for pid in TRAIN_IDS:
    PREP_DIR = os.path.join(BASE_PREP_ROOT, pid)
    print("\n" + "="*90)
    print("Checking preprocessed outputs for:", pid)
    print("Folder:", PREP_DIR)

    f_path = os.path.join(PREP_DIR, "followup_0p5_norm.nii.gz")
    b_path = os.path.join(PREP_DIR, "baseline_0p5_registered_to_followup_norm.")
    gt_path = os.path.join(PREP_DIR, "label_0p5_in_followup_space.nii.gz")

    # --- check files exist ---
    missing = [p for p in [f_path, b_path, gt_path] if not os.path.exists(p)]
    if missing:
        print("⚠ Missing files, skipping:")
        for m in missing:
            print(" -", os.path.basename(m))
        continue

    # --- load volumes ---
    f = nib.load(f_path).get_fdata().astype(np.float32)
    b = nib.load(b_path).get_fdata().astype(np.float32)
    gt = nib.load(gt_path).get_fdata().astype(np.float32)

    gt_bin = gt > 0.5

    print("Shapes (F, B, GT):", f.shape, b.shape, gt_bin.shape)
    print("GT voxels:", int(gt_bin.sum()))

    # pick slices that contain GT
    z_sums = gt_bin.sum(axis=(1, 2))
    idx = np.where(z_sums > 0)[0]

    if idx.size == 0:
        print("⚠ No GT voxels after warping/resampling for this patient.")
        continue

    # choose up to 3 representative slices
    chosen = np.linspace(idx.min(), idx.max(), 3).astype(int)
    print("Showing slices:", chosen.tolist())

    for sl in chosen:
        fig, ax = plt.subplots(1, 3, figsize=(12, 4))

        ax[0].imshow(f[sl, :, :], cmap="gray")
        ax[0].set_title(f"{pid} Follow-up (slice {sl})")
        ax[0].axis("off")

```

```

ax[1].imshow(b[s1, :, :], cmap="gray")
ax[1].set_title(f"{pid} Baseline reg→Follow-up (slice {s1})")
ax[1].axis("off")

# overlay GT on follow-up
ax[2].imshow(f[s1, :, :], cmap="gray")
ax[2].imshow(gt_bin[s1, :, :], alpha=0.5, cmap="Reds")
ax[2].set_title(f"{pid} Follow-up + GT overlay")
ax[2].axis("off")

plt.tight_layout()
plt.show()

```

Conclusion for above :

- Baseline scans were rigidly registered to follow-up scans.
- All images were resampled to 0.5 mm isotropic resolution, intensity-normalized, and padded to a common spatial size.
- Ground-truth lesion masks were transformed into the follow-up reference space

Step 5.2: Preprocess Test Patients

```

TEST_IDS = ["00960975", "00981157"]

for pid in TEST_IDS:
    print("\n" + "="*80)
    print("Preprocessing TEST patient:", pid)

    out_dir = os.path.join(OUT_ROOT, pid)
    os.makedirs(out_dir, exist_ok=True)

    baseline_p, followup_p, label_p = get_patient_paths(pid)

    b = sitk_read(baseline_p)
    f = sitk_read(followup_p)
    gt = sitk_read(label_p)

    # Register baseline → follow-up
    tfm = register_baseline_to_followup(b, f)
    b_reg = sitk.Resample(b, f, tfm, sitk.sitkLinear, 0.0, b.GetPixelID())

    # 0.5mm reference grid in follow-up space
    ref05 = make_reference_with_spacing(f, TARGET_SPACING)

```

```

# Resample images
f_05 = sitk.Resample(f, ref05, sitk.Transform(), sitk.sitkLinear, 0.0, f.GetSpacing())
b_05 = sitk.Resample(b_reg, ref05, sitk.Transform(), sitk.sitkLinear, 0.0, b_reg.GetSpacing())

# Warp GT → follow-up, then resample
gt_to_followup = sitk.Resample(gt, f, tfm, sitk.sitkNearestNeighbor, 0, gt.GetSpacing())
gt_05 = sitk.Resample(gt_to_followup, ref05, sitk.Transform(), sitk.sitkNearestNeighbor, 0, gt_05.GetSpacing())

# Normalize (no N4)
mask = simple_foreground_mask(f_05)
f_norm = zscore_in_mask(f_05, mask)
b_norm = zscore_in_mask(b_05, mask)

# Save
sitk.WriteImage(b_norm, os.path.join(out_dir, "baseline_0p5_registered_to_followup.nii.gz"))
sitk.WriteImage(f_norm, os.path.join(out_dir, "followup_0p5_norm.nii.gz"))
sitk.WriteImage(gt_05, os.path.join(out_dir, "label_0p5_in_followup_space.nii.gz"))

print("Saved to:", out_dir)

print("\nAll TEST patients preprocessed ✅")

```

```

=====
Preprocessing TEST patient: 00960975
Saved to: /content/drive/MyDrive/ms_preprocessed/00960975

=====
Preprocessing TEST patient: 00981157
Saved to: /content/drive/MyDrive/ms_preprocessed/00981157

All TEST patients preprocessed ✅

```

Step 6: Patch Dataset Construction (Training Only)

build a patch dataset + dataloader from the preprocessed training patient

Input patch = (2, D, H, W) (baseline + followup as channels)

Target patch = (1, D, H, W) (new lesions)

Balanced sampling: 50% positive patches, 50% negative patches

Then we train Pipeline #1: Modified 3D U-Net

Step 6.1: Install & Import Deep Learning Libraries

```
!pip -q install torch torchvision torchaudio --index-url https://download.pytor
```

```
import os
import numpy as np
import nibabel as nib
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
```

Step 6.2: Patch Dataset Parameters and Balanced Patch Dataset (On-the-Fly Sampling)

```
import os
import numpy as np
import nibabel as nib
import torch
from torch.utils.data import Dataset, DataLoader

# =====
# MULTI-PATIENT PATCH TRAINING
# =====

PREP_ROOT = "/content/drive/MyDrive/ms_preprocessed"

TRAIN_IDS = [
    "00713519",
    "01092669",
    "01386182",
    "01406050",
    "01722254",
    "01406050_1",
    "01563167",
    "01581148_2",
    "01581148_4",
    "01668999",
]

PATCH_SIZE = (64, 64, 64)
SAMPLES_PER_EPOCH = 800
    # total samples across ALL patients per epoch
POS_FRACTION = 0.5          # 50% positive patches

def load_preprocessed(pid):
    pdir = os.path.join(PREP_ROOT, pid)
    b = nib.load(os.path.join(pdir, "baseline_0p5_registered_to_followup_norm.r
    f = nib.load(os.path.join(pdir, "followup_0p5_norm.nii.gz")).get_fdata().as
```

```

y = nib.load(os.path.join(pdir, "label_0p5_in_followup_space.nii.gz")).get_
y = (y > 0.5).astype(np.uint8)
return b, f, y

def valid_patch_bounds(shape, patch):
    D, H, W = shape
    pd, ph, pw = patch
    if D < pd or H < ph or W < pw:
        raise ValueError(f"Patch {patch} bigger than volume shape {shape}")
    return (D - pd, H - ph, W - pw)

class MultiPatientBalancedPatchDataset(Dataset):
    """
    Each __getitem__:
        1) randomly selects a patient
        2) samples a pos/neg patch from that patient
    """
    def __init__(self, patient_ids, patch_size=(64,64,64), samples_per_epoch=4096):
        self.pids = list(patient_ids)
        self.patch = patch_size
        self.N = samples_per_epoch
        self.pos_fraction = pos_fraction

        # Load all patients into memory (fast sampling, but can be RAM heavy)
        self.data = {}
        for pid in self.pids:
            b, f, y = load_preprocessed(pid)

            pos_coords = np.argwhere(y > 0)
            has_pos = (pos_coords.shape[0] > 0)
            maxz, maxy, maxx = valid_patch_bounds(b.shape, self.patch)

            self.data[pid] = {
                "b": b, "f": f, "y": y,
                "pos_coords": pos_coords,
                "has_pos": has_pos,
                "maxz": maxz, "maxy": maxy, "maxx": maxx,
                "shape": b.shape
            }

            # Optional: print a quick summary
            print("Loaded patients into dataset:")
            for pid in self.pids:
                d = self.data[pid]
                print(f" - {pid}: shape={d['shape']}, GT_voxels={int(d['y'].sum())}")

        def __len__(self):
            return self.N

        def _random_start(self, maxz, maxy, maxx):
            z = np.random.randint(0, maxz + 1)

```

```

        y = np.random.randint(0, maxy + 1)
        x = np.random.randint(0, maxx + 1)
        return z, y, x

def _pos_start_centered(self, pos_coords, maxz, maxy, maxx):
    cz, cy, cx = pos_coords[np.random.randint(0, len(pos_coords))]
    pd, ph, pw = self.patch
    z = int(np.clip(cz - pd//2, 0, maxz))
    y = int(np.clip(cy - ph//2, 0, maxy))
    x = int(np.clip(cx - pw//2, 0, maxx))
    return z, y, x

def __getitem__(self, idx):
    # 1) choose a patient
    pid = self.pids[np.random.randint(0, len(self.pids))]
    d = self.data[pid]

    # 2) choose pos or neg patch
    want_pos = (np.random.rand() < self.pos_fraction) and d["has_pos"]
    if want_pos:
        z, y0, x = self._pos_start_centered(d["pos_coords"], d["maxz"], d["maxy"], d["maxx"])
    else:
        z, y0, x = self._random_start(d["maxz"], d["maxy"], d["maxx"])

    pd, ph, pw = self.patch
    b_patch = d["b"][z:z+pd, y0:y0+ph, x:x+pw]
    f_patch = d["f"][z:z+pd, y0:y0+ph, x:x+pw]
    y_patch = d["y"][z:z+pd, y0:y0+ph, x:x+pw]

    X = np.stack([b_patch, f_patch], axis=0)          # (2, D, H, W)
    Y = y_patch[None, ...].astype(np.float32)        # (1, D, H, W)

    return torch.from_numpy(X), torch.from_numpy(Y)

# Create dataset + loader
train_ds = MultiPatientBalancedPatchDataset(
    TRAIN_IDS,
    patch_size=PATCH_SIZE,
    samples_per_epoch=SAMPLES_PER_EPOCH,
    pos_fraction=POS_FRACTION
)

train_loader = DataLoader(
    train_ds,
    batch_size=2,
    shuffle=True,
    num_workers=0,
    pin_memory=True
)

# Sanity check batch shapes (don't call next() twice)

```

```
Xb, Yb = next(iter(train_loader))
print("Example batch shapes:", Xb.shape, Yb.shape) # expected: (B,2,D,H,W) and
```

Loaded patients into dataset:

- 00713519: shape=(330, 500, 500), GT_voxels=7940
- 01092669: shape=(501, 501, 302), GT_voxels=3498
- 01386182: shape=(330, 500, 500), GT_voxels=33515
- 01406050: shape=(330, 500, 500), GT_voxels=96504
- 01722254: shape=(365, 500, 500), GT_voxels=2770
- 01406050_1: shape=(500, 500, 365), GT_voxels=25363
- 01563167: shape=(365, 500, 500), GT_voxels=4496
- 01581148_2: shape=(365, 500, 500), GT_voxels=5432
- 01581148_4: shape=(354, 500, 500), GT_voxels=2546
- 01668999: shape=(330, 500, 500), GT_voxels=686

Example batch shapes: torch.Size([2, 2, 64, 64, 64]) torch.Size([2, 1, 64, 64, 64])

Step 7: Model and Training (NeuroPoly Pipeline #1)

Step 7.1: Model Architecture — Modified 3D U-Net (Pipeline #1)

```
class ConvBlock(nn.Module):
    def __init__(self, in_ch, out_ch):
        super().__init__()
        self.conv1 = nn.Conv3d(in_ch, out_ch, 3, padding=1)
        self.in1 = nn.InstanceNorm3d(out_ch)
        self.conv2 = nn.Conv3d(out_ch, out_ch, 3, padding=1)
        self.in2 = nn.InstanceNorm3d(out_ch)
        self.act = nn.LeakyReLU(0.01, inplace=True)

    def forward(self, x):
        x = self.act(self.in1(self.conv1(x)))
        x = self.act(self.in2(self.conv2(x)))
        return x

class Down(nn.Module):
    def __init__(self, in_ch, out_ch):
        super().__init__()
        self.down = nn.Conv3d(in_ch, out_ch, kernel_size=3, stride=2, padding=1)
        self.in0 = nn.InstanceNorm3d(out_ch)
        self.act = nn.LeakyReLU(0.01, inplace=True)
        self.block = ConvBlock(out_ch, out_ch)

    def forward(self, x):
        x = self.act(self.in0(self.down(x)))
        x = self.block(x)
        return x

class Up(nn.Module):
```

```

def __init__(self, in_ch, skip_ch, out_ch):
    super().__init__()
    self.up = nn.Upsample(scale_factor=2, mode="nearest")
    self.conv1x1 = nn.Conv3d(in_ch, out_ch, kernel_size=1)      # match din
    self.skip1x1 = nn.Conv3d(skip_ch, out_ch, kernel_size=1)   # match ski
    self.block = ConvBlock(out_ch*2, out_ch)

def forward(self, x, skip):
    x = self.up(x)
    x = self.conv1x1(x)
    skip = self.skip1x1(skip)
    x = torch.cat([skip, x], dim=1)
    return self.block(x)

class ModifiedUNet3D(nn.Module):
    def __init__(self, in_ch=2, base=16):
        super().__init__()
        self.enc0 = ConvBlock(in_ch, base)
        self.enc1 = Down(base, base*2)
        self.enc2 = Down(base*2, base*4)
        self.enc3 = Down(base*4, base*8)

        self.bottleneck = Down(base*8, base*16)

        self.dec3 = Up(base*16, base*8, base*8)
        self.dec2 = Up(base*8, base*4, base*4)
        self.dec1 = Up(base*4, base*2, base*2)
        self.dec0 = Up(base*2, base, base)

        self.out = nn.Conv3d(base, 1, kernel_size=1)

    def forward(self, x):
        s0 = self.enc0(x)
        s1 = self.enc1(s0)
        s2 = self.enc2(s1)
        s3 = self.enc3(s2)
        b = self.bottleneck(s3)
        x = self.dec3(b, s3)
        x = self.dec2(x, s2)
        x = self.dec1(x, s1)
        x = self.dec0(x, s0)
        return self.out(x) # logits

```

```

import numpy as np
import torch

# ----- Dice loss (train) -----
def soft_dice_loss(logits, targets, eps=1e-6):
    probs = torch.sigmoid(logits)

```

```

    num = 2 * (probs * targets).sum(dim=(2, 3, 4))
    den = (probs + targets).sum(dim=(2, 3, 4)) + eps
    dice = num / den
    return 1 - dice.mean()

# ----- Dice score (eval metric) -----
def dice_score(pred, gt, eps=1e-8):
    pred = pred.astype(bool)
    gt = gt.astype(bool)
    inter = np.logical_and(pred, gt).sum()
    denom = pred.sum() + gt.sum()
    return (2.0 * inter + eps) / (denom + eps)

print("✅ Loss and metric functions loaded")

```

✅ Loss and metric functions loaded

```

import numpy as np
import torch

# -----
# Device
# -----
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("✅ Using device:", device)

# -----
# Loss (train)
# -----
def soft_dice_loss(logits, targets, eps=1e-6):
    probs = torch.sigmoid(logits)
    num = 2 * (probs * targets).sum(dim=(2, 3, 4))
    den = (probs + targets).sum(dim=(2, 3, 4)) + eps
    dice = num / den
    return 1 - dice.mean()

# -----
# Metric (eval)
# -----
def dice_score(pred, gt, eps=1e-8):
    pred = pred.astype(bool)
    gt = gt.astype(bool)
    inter = np.logical_and(pred, gt).sum()
    denom = pred.sum() + gt.sum()
    return (2.0 * inter + eps) / (denom + eps)

# -----
# Test patient IDs (edit if needed)
# -----
TEST_IDS = ["00960975", "00981157"]

```

```

# -----
# Sliding-window inference
# -----
def sliding_window_predict(model, b, f, patch_size=(64,64,64), stride=(32,32,32), device='cpu'):
    model.eval()
    D, H, W = b.shape
    pd, ph, pw = patch_size
    sd, sh, sw = stride

    prob = np.zeros((D, H, W), dtype=np.float32)
    count = np.zeros((D, H, W), dtype=np.float32)

    with torch.no_grad():
        for z in range(0, D - pd + 1, sd):
            for y in range(0, H - ph + 1, sh):
                for x in range(0, W - pw + 1, sw):
                    b_patch = b[z:z+pd, y:y+ph, x:x+pw]
                    f_patch = f[z:z+pd, y:y+ph, x:x+pw]

                    X = np.stack([b_patch, f_patch], axis=0)[None] # (1,2,D,H,W)
                    X = torch.from_numpy(X).to(device)

                    logits = model(X)
                    p = torch.sigmoid(logits)[0, 0].detach().cpu().numpy()

                    prob[z:z+pd, y:y+ph, x:x+pw] += p
                    count[z:z+pd, y:y+ph, x:x+pw] += 1

    count[count == 0] = 1
    return prob / count

# -----
# Evaluate on test patients
# -----
def evaluate_on_tests(model, thr=0.5):
    model.eval()
    results = {}

    with torch.no_grad():
        for pid in TEST_IDS:
            b, f, gt = load_preprocessed(pid) # MUST exist from your dataset 1

            prob = sliding_window_predict(
                model, b, f,
                patch_size=PATCH_SIZE,
                stride=STRIDE
            )
            pred = (prob >= thr).astype(np.uint8)
            results[pid] = float(dice_score(pred, gt))

    model.train()

```

```

        return results

print("✅ Common functions ready: soft_dice_loss, dice_score, sliding_window_p

✅ Using device: cuda
✅ Common functions ready: soft_dice_loss, dice_score, sliding_window_predict,

```

```

PATCH_SIZE = (64, 64, 64)
STRIDE = (32, 32, 32)

print("PATCH_SIZE:", PATCH_SIZE)
print("STRIDE:", STRIDE)

```

```

PATCH_SIZE: (64, 64, 64)
STRIDE: (32, 32, 32)

```

Step 7.2-A: 500 epochs training for best grid search combo

```

import os, json
import numpy as np
import torch

# =====
# FINAL TRAIN SETTINGS (LONG)
# =====
MAX_EPOCHS = 500
THR = 0.5

BEST_LR = 3e-4
BEST_WD = 1e-6

# Evaluate & save frequency
EVAL_EVERY = 10          # eval every 10 epochs (change if slow)
SAVE_EVERY = 50          # extra snapshot every 50 epochs

RUN_NAME = f"final_train10_ep{MAX_EPOCHS}_lr{BEST_LR}_wd{BEST_WD}"
OUT_ROOT = f"/content/drive/MyDrive/ms_preprocessed/_final/{RUN_NAME}"
os.makedirs(OUT_ROOT, exist_ok=True)

BEST_CKPT_PATH = os.path.join(OUT_ROOT, "best.pt")
LAST_CKPT_PATH = os.path.join(OUT_ROOT, "last.pt")
HISTORY_PATH = os.path.join(OUT_ROOT, "history.json")

# Optional: resume from your best grid checkpoint (epoch 25)
RESUME = True
RESUME_CKPT = "/content/drive/MyDrive/ms_preprocessed/_grid/grid_train10_ep25/1

def avg_test_dice(scores_dict):
    vals = [float(v) for v in scores_dict.values()]

```

```

        return float(np.mean(vals))

# Reproducibility (optional)
torch.manual_seed(0)
np.random.seed(0)

# =====
# Model + Optimizer
# =====
model = ModifiedUNet3D(in_ch=2, base=16).to(device)
opt = torch.optim.AdamW(model.parameters(), lr=BEST_LR, weight_decay=BEST_WD)

start_epoch = 1
best_avg = -1.0
history = []

if RESUME and os.path.exists(RESUME_CKPT):
    ckpt = torch.load(RESUME_CKPT, map_location=device)
    model.load_state_dict(ckpt["model_state"])
    # You can resume optimizer too (optional). If you want exact continuation:
    try:
        opt.load_state_dict(ckpt["opt_state"])
    except:
        print("⚠️ Could not load optimizer state, continuing with fresh optimi
        start_epoch = int(ckpt.get("epoch", 0)) + 1
        best_avg = float(ckpt.get("avg_test_dice", -1.0))
        print(f"✅ Resumed from {RESUME_CKPT} at epoch {start_epoch-1}, best_avg={

# =====
# Training Loop
# =====
model.train()

for epoch in range(start_epoch, MAX_EPOCHS + 1):
    losses = []

    for X, Y in train_loader:
        X = X.to(device, non_blocking=True)
        Y = Y.to(device, non_blocking=True)

        opt.zero_grad()
        logits = model(X)
        loss = soft_dice_loss(logits, Y)
        loss.backward()
        opt.step()

        losses.append(loss.item())

    mean_loss = float(np.mean(losses))

    if epoch % 5 == 0 or epoch == 1:

```

```

        print(f"Epoch {epoch:3d} | train Dice-loss: {mean_loss:.4f}")

# -----
# Evaluation
# -----
do_eval = (epoch % EVAL_EVERY == 0) or (epoch == MAX_EPOCHS)
if do_eval:
    model.eval()
    with torch.no_grad():
        scores = evaluate_on_tests(model, thr=THR)
    avg_d = avg_test_dice(scores)
    model.train()

    print(f"  ✅ Test Dice @ epoch {epoch}: {scores} | avg={avg_d:.4f}")

    row = {
        "epoch": int(epoch),
        "train_dice_loss": float(mean_loss),
        "test_dice": {k: float(v) for k, v in scores.items()},
        "avg_test_dice": float(avg_d),
        "lr": float(BEST_LR),
        "weight_decay": float(BEST_WD),
    }
    history.append(row)

# Save BEST checkpoint (by avg test dice)
if avg_d > best_avg:
    best_avg = avg_d
    torch.save({
        "epoch": epoch,
        "model_state": model.state_dict(),
        "opt_state": opt.state_dict(),
        "lr": BEST_LR,
        "weight_decay": BEST_WD,
        "train_dice_loss": mean_loss,
        "test_dice": {k: float(v) for k, v in scores.items()},
        "avg_test_dice": float(avg_d),
    }, BEST_CKPT_PATH)
    print(f"  🏆 New BEST saved: {BEST_CKPT_PATH} (avg={best_avg:.4f})

# Save history continuously
with open(HISTORY_PATH, "w") as f:
    json.dump(history, f, indent=2)

# -----
# Periodic snapshot + last
# -----
if epoch % SAVE_EVERY == 0:
    snap_path = os.path.join(OUT_ROOT, f"snapshot_epoch{epoch}.pt")
    torch.save({
        "epoch": epoch,

```

```

        "model_state": model.state_dict(),
        "opt_state": opt.state_dict(),
        "lr": BEST_LR,
        "weight_decay": BEST_WD,
        "train_dice_loss": mean_loss,
    }, snap_path)
    print("💾 Snapshot saved:", snap_path)

# Always keep a "last" checkpoint
torch.save({
    "epoch": epoch,
    "model_state": model.state_dict(),
    "opt_state": opt.state_dict(),
    "lr": BEST_LR,
    "weight_decay": BEST_WD,
    "train_dice_loss": mean_loss,
    "best_avg_so_far": float(best_avg),
}, LAST_CKPT_PATH)

print("\n✅ Training finished.")
print("Best checkpoint:", BEST_CKPT_PATH)
print("Last checkpoint:", LAST_CKPT_PATH)
print("History:", HISTORY_PATH)

```

```

✅ Resumed from /content/drive/MyDrive/ms_preprocessed/_grid/grid_train10_ep25
Epoch 30 | train Dice-loss: 0.5602
✅ Test Dice @ epoch 30: {'00960975': 0.008177722119663156, '00981157': 0.07
Epoch 35 | train Dice-loss: 0.5640
Epoch 40 | train Dice-loss: 0.5688
✅ Test Dice @ epoch 40: {'00960975': 0.009932804825176505, '00981157': 0.09
Epoch 45 | train Dice-loss: 0.5785
Epoch 50 | train Dice-loss: 0.5600
✅ Test Dice @ epoch 50: {'00960975': 0.01858190709050982, '00981157': 0.226
💾 Snapshot saved: /content/drive/MyDrive/ms_preprocessed/_final/final_train
Epoch 55 | train Dice-loss: 0.5453
Epoch 60 | train Dice-loss: 0.5603
✅ Test Dice @ epoch 60: {'00960975': 0.01263276944664915, '00981157': 0.156
Epoch 65 | train Dice-loss: 0.5257
Epoch 70 | train Dice-loss: 0.5623
✅ Test Dice @ epoch 70: {'00960975': 0.01874230375893729, '00981157': 0.219
Epoch 75 | train Dice-loss: 0.5401
Epoch 80 | train Dice-loss: 0.5258
✅ Test Dice @ epoch 80: {'00960975': 0.023349373699042074, '00981157': 0.26
🏆 New BEST saved: /content/drive/MyDrive/ms_preprocessed/_final/final_train
Epoch 85 | train Dice-loss: 0.5243
Epoch 90 | train Dice-loss: 0.5220
✅ Test Dice @ epoch 90: {'00960975': 0.01744111329383622, '00981157': 0.227
Epoch 95 | train Dice-loss: 0.5325
Epoch 100 | train Dice-loss: 0.5402
✅ Test Dice @ epoch 100: {'00960975': 0.010453738127042032, '00981157': 0.1
💾 Snapshot saved: /content/drive/MyDrive/ms_preprocessed/_final/final_train
Epoch 105 | train Dice-loss: 0.5213
Epoch 110 | train Dice-loss: 0.5181
✅ Test Dice @ epoch 110: {'00960975': 0.020731099023822676, '00981157': 0.2

```

```

🏆 New BEST saved: /content/drive/MyDrive/ms_preprocessed/_final/final_train
Epoch 115 | train Dice-loss: 0.5292
Epoch 120 | train Dice-loss: 0.5041
✅ Test Dice @ epoch 120: {'00960975': 0.020498962424222427, '00981157': 0.2
Epoch 125 | train Dice-loss: 0.5069
Epoch 130 | train Dice-loss: 0.5238
✅ Test Dice @ epoch 130: {'00960975': 0.01682214916310572, '00981157': 0.25
Epoch 135 | train Dice-loss: 0.4845
Epoch 140 | train Dice-loss: 0.5016
✅ Test Dice @ epoch 140: {'00960975': 0.015297636739655113, '00981157': 0.1
Epoch 145 | train Dice-loss: 0.5136
Epoch 150 | train Dice-loss: 0.5331
✅ Test Dice @ epoch 150: {'00960975': 0.017321347635631614, '00981157': 0.2
💾 Snapshot saved: /content/drive/MyDrive/ms_preprocessed/_final/final_train
Epoch 155 | train Dice-loss: 0.4961
Epoch 160 | train Dice-loss: 0.5080
✅ Test Dice @ epoch 160: {'00960975': 0.01935572398321834, '00981157': 0.23
Epoch 165 | train Dice-loss: 0.5112
Epoch 170 | train Dice-loss: 0.5415
✅ Test Dice @ epoch 170: {'00960975': 0.020815007546411503, '00981157': 0.2
🏆 New BEST saved: /content/drive/MyDrive/ms_preprocessed/_final/final_train
Epoch 175 | train Dice-loss: 0.4938
Epoch 180 | train Dice-loss: 0.4836
✅ Test Dice @ epoch 180: {'00960975': 0.023029318753863534, '00981157': 0.2
🏆 New BEST saved: /content/drive/MyDrive/ms_preprocessed/_final/final_train
Epoch 185 | train Dice-loss: 0.4820
Epoch 190 | train Dice-loss: 0.5235

```

Step 7.3: Visual Evaluation — Ground Truth vs Prediction

```

import os
import numpy as np
import torch
import matplotlib.pyplot as plt

# =====
# VISUALIZATION FOR FINAL 500-EPOCH TRAINING
# =====

# 🔥 CHANGE THIS to your final training folder
RUN_NAME = "final_train10_ep500_lr0.0003_wd1e-06"

BASE_PATH = f"/content/drive/MyDrive/ms_preprocessed/_final/{RUN_NAME}"

# Which epochs you want to visualize
EPOCHS_TO_VIEW = [50, 100, 200, 250, 500]

THR = 0.5
TEST_PATIENTS = ["00960975", "00981157"]

OUT_VIS_DIR = os.path.join(BASE_PATH, "visualizations")
os.makedirs(OUT_VIS_DIR, exist_ok=True)

```

```

# -----
# Utility: pick best slice
# -----
def best_slice_from_gt(gt, pred=None):
    z_gt = gt.sum(axis=(1, 2))
    if z_gt.max() > 0:
        return int(np.argmax(z_gt))
    if pred is not None:
        z_pr = pred.sum(axis=(1, 2))
        if z_pr.max() > 0:
            return int(np.argmax(z_pr))
    return gt.shape[0] // 2

# -----
# Plot function
# -----
def plot_epoch(epoch, thr=0.5):

    # If you saved snapshots like snapshot_epoch50.pt
    CKPT_PATH = os.path.join(BASE_PATH, f"snapshot_epoch{epoch}.pt")

    # For final epoch you may want best.pt or last.pt
    if epoch == 500 and not os.path.exists(CKPT_PATH):
        CKPT_PATH = os.path.join(BASE_PATH, "last.pt")

    if not os.path.exists(CKPT_PATH):
        print(f"⚠️ Checkpoint not found for epoch {epoch}")
        return

    ckpt = torch.load(CKPT_PATH, map_location=device)
    model.load_state_dict(ckpt["model_state"])
    model.eval()

    print("✅ Loaded:", CKPT_PATH)

    fig, axes = plt.subplots(len(TEST_PATIENTS), 2, figsize=(7, 3 * len(TEST_PATIENTS)))
    if len(TEST_PATIENTS) == 1:
        axes = np.array([axes])

    for i, pid in enumerate(TEST_PATIENTS):

        b, f, gt = load_preprocessed(pid)

        prob = sliding_window_predict(model, b, f,
                                      patch_size=PATCH_SIZE,
                                      stride=STRIDE)

        pred = (prob >= thr).astype(np.uint8)
        d = dice_score(pred, gt)

```

```

sl = best_slice_from_gt(gt, pred)

axes[i, 0].imshow(gt[sl], cmap="gray")
axes[i, 0].set_title(f"GT\nPatient {pid}")
axes[i, 0].axis("off")

axes[i, 1].imshow(pred[sl], cmap="gray")
axes[i, 1].set_title(f"Pred\nPatient {pid}\nDice={d:.4f}")
axes[i, 1].axis("off")

plt.suptitle(f"{RUN_NAME} – Epoch {epoch} (thr={thr})", fontsize=13)
plt.tight_layout(rect=[0, 0, 1, 0.95])

save_path = os.path.join(OUT_VIS_DIR, f"epoch_{epoch}_thr{thr}.png")
plt.savefig(save_path, dpi=200, bbox_inches="tight")
plt.show()

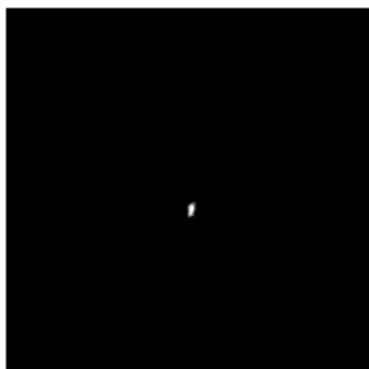
print("💾 Saved:", save_path)

# -----
# Loop over requested epochs
# -----
for ep in EPOCHS_TO_VIEW:
    plot_epoch(ep, thr=THR)

```

✓ Loaded: /content/drive/MyDrive/ms_preprocessed/_final/final_train10_ep500_1
final_train10_ep500_lr0.0003_wd1e-06 — Epoch 50 (thr=0.5)

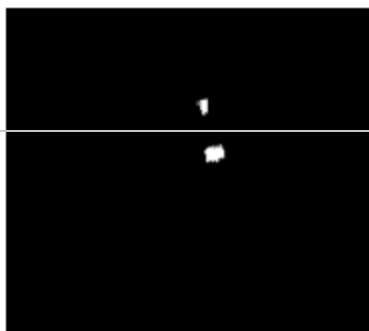
GT
Patient 00960975



Pred
Patient 00960975
Dice=0.0186



GT
Patient 00981157



Pred
Patient 00981157
Dice=0.2204

